Référence complète – `class GITGlobalModel` (uniquement ce qui est dans le code)

1) Paramètres d'entrée (arguments de `__init__`)

- `C=CONST` : dictionnaire de constantes physiques, utilisées via `self.C['...']`.
- `L_chamber=0.08` : L [m]
- `A_chamber=1e-3` : A [m^2]
- `mass_flow=2e-6` : $\dot{m}$ [kg/s]
- `T_neutrals=500` : T_n [K]
- `V_beam=800.0` : V_{beam} [V]
- `d_grids=1.2e-3` : d [m]
- `tau_optical=0.65` : τ [-]
- `Te_eV=5.0` : T_e [eV]
- `ki_coeff=1e-14` : k_i [m^3/s]
- `P_discharge=10.0` : P_{dis} [W]

2) Variables stockées (attributs `self.*`)

- Géométrie / entrée : `self.L, self.A, self.mass_flow, self.T_neutrals, self.V_beam, self.d_grids, self.tau, self.Te_eV, self.ki, self.P_discharge`.
- Dérivées géométriques : `self.V_ch, self.R`.
- Vitesses : `self.u_B, self.v_neutrals`.
- Pertes parois : `self.lambda_i, self.h_L, self.h_R, self.A_eff_wall, self.nu_loss_vol`.
- Débits particules : `self.N_dot_neutral, self.N_dot_beam, self.N_dot_wall`.
- Courants : `self.I_CL, self.I_plasma_available, self.I_wall_loss, self.I_beam`.
- Densités : `self.n_a, self.n_i`.
- Rendements stockés : `self.eta_mass_total, self.eta_mass_beam` (et `self.eta_mass` existe mais n'est pas recalculé ensuite).
- Autres : `self.fluxes` (initialisé à {} mais non utilisé dans les calculs).

3) Calculs effectués dans `__init__` (dans l'ordre du code)

3.1 Volume de chambre

$$V_{ch} = L\,A$$

(`self.V_ch = self.L * self.A`)

3.2 Vitesse de Bohm

$$u_B = \sqrt{\frac{e\,T_e}{m_i}}$$

(`self.u_B = sqrt(self.C['e'] * self.Te_eV / self.m_i)`)

3.3 Vitesse thermique des neutres

$$v_n = \sqrt{\frac{8k_B T_n}{\pi m_i}}$$

(`self.v_neutrals = sqrt(8*self.C['kB']*self.T_neutrals/(pi*self.m_i))`)

3.4 Rayon équivalent (depuis la section)

$$R = \sqrt{\frac{A}{\pi}}$$

(`self.R = sqrt(self.A/pi)`)

3.5 Longueur libre moyenne (constante dans le code)

$$\lambda_i = 0.03 \text{ m}$$

(`self.lambda_i = 0.03`)

3.6 Edge-to-center ratios (Goebel & Katz) comme dans le code

$$h_L = 0.86\left(3 + \frac{L}{2\lambda_i}\right)^{-1/2} \qquad h_R = 0.8\left(4 + \frac{R}{\lambda_i}\right)^{-1/2}$$

(`self.h_L = 0.86*(3+L/(2*lambda_i))**(-0.5)`, `self.h_R = 0.8*(4+R/lambda_i)**(-0.5)`)

3.7 Aire effective de pertes parois

$$A_{\text{eff,wall}} = 2h_R \pi R L + 2h_L \pi R^2$$

(`self.A_eff_wall = 2*h_R*pi*R*L + 2*h_L*pi*R**2`)

3.8 Fréquence de perte volumique

$$\nu_{\text{loss,vol}} = \frac{u_B A_{\text{eff,wall}}}{V_{ch}}$$

(`self.nu_loss_vol = (u_B*A_eff_wall)/V_ch`)

3.9 Flux particulaire neutre entrant (particules/s)

$$\dot{N}_n = \frac{\dot{m}}{m_i}$$

(`self.N_dot_neutral = self.mass_flow / self.m_i`)

3.10 Courant limite Child–Langmuir (appel méthode)

$$I_{CL} = \text{self._calculate_I_CL()}$$

(`self.I_CL = self._calculate_I_CL()`)

4) Calculs dans `_calculate_I_CL()` (uniquement ce qui est dans le code)

Le code calcule d'abord une densité de courant :

$$J_{CL} = \frac{4}{9}\,\varepsilon_0 \sqrt{\frac{2e}{m_i}}\,\frac{V_{\text{beam}}^{1.5}}{d^2}$$

(`J_CL = (4/9)*eps0*sqrt(2*e/m_i)*(V_beam**1.5)/(d_grids**2)`)

Puis le courant limite total :

$$I_{CL} = J_{CL}\,A\,\tau$$

(`I_CL = J_CL * self.A * self.tau`)

5) Calculs dans `solve_equations()` (ordre exact du code)

5.1 Densité neutre d'équilibre

Le code impose $\nu_{iz} = \nu_{\text{loss,vol}}$ avec $\nu_{iz} = k_i n_a$, donc :

$$n_a = \frac{\nu_{\text{loss,vol}}}{k_i}$$

(`n_a_eq = nu_loss_vol / ki`, puis `self.n_a = n_a_eq`)

5.2 Densité ionique (heuristique fixée)

$$n_i = 0.01\, n_a$$

(`self.n_i = self.n_a * 0.01`)

5.3 Courant ionique "disponible plasma" (nom exact du code)

$$I_{\text{plasma,available}} = e\, n_i\, u_B\, A$$

(`self.I_plasma_available = e*n_i*u_B*A`)

5.4 Courant de pertes aux parois

$$I_{\text{wall,loss}} = e\, n_i\, u_B\, A_{\text{eff,wall}}$$

(`self.I_wall_loss = e*n_i*u_B*A_eff_wall`)

5.5 Courant de faisceau extrait

Le code prend le minimum entre :

- la limite Child–Langmuir I_{CL},
- et le courant "disponible plasma" multiplié par la transparence τ.

$$I_{\text{beam}} = \min\left(I_{CL},\ \tau\, I_{\text{plasma,available}}\right)$$

(`self.I_beam = min(self.I_CL, self.I_plasma_available * self.tau)`)

5.6 Bilans en débits particulaires d'ions

$$\dot{N}_{\text{beam}} = \frac{I_{\text{beam}}}{e} \qquad \dot{N}_{\text{wall}} = \frac{I_{\text{wall,loss}}}{e}$$

(`self.N_dot_beam = I_beam/e`, `self.N_dot_wall = I_wall_loss/e`)

6) Calculs dans `compute_and_print_performances()` (ordre exact du code)

6.1 Condition d'arrêt

Si $I_{\text{beam}} = 0$: arrêt (pas de performances).

6.2 Vitesse faisceau ionique

$$v_{\text{beam}} = \sqrt{\frac{2eV_{\text{beam}}}{m_i}}$$

(`v_beam = sqrt(2*e*V_beam/m_i)`)

6.3 Poussée

$$T = I_{\text{beam}} \frac{m_i v_{\text{beam}}}{e}$$

(`T_thrust = I_beam * m_i * v_beam / e`)

6.4 Impulsions spécifiques (2 définitions dans le code)

$$I_{sp,global} = \frac{T}{\dot{m}\, g_0} \qquad I_{sp,beam} = \frac{v_{\text{beam}}}{g_0}$$

(`Isp_global = T_thrust/(mass_flow*g0)` et `Isp_beam = v_beam/g0`)

Différence (comme le dit le commentaire du code) :

- $I_{sp,global}$ utilise $\dot{m}$ total injecté $\Rightarrow$ *pénalisé* par les neutres non accélérés.
- $I_{sp,beam}$ est basé uniquement sur la vitesse ionique $\Rightarrow$ *faisceau uniquement.*

6.5 Puissances

$$P_{\text{beam}} = I_{\text{beam}} V_{\text{beam}} \qquad P_{\text{total}} = P_{\text{beam}} + P_{\text{dis}}$$

(`P_beam = I_beam*V_beam`, `P_total = P_beam + P_discharge`)

6.6 Puissance jet (définition exacte utilisée par le code)

$$P_{\text{jet}} = 0.5\, T\, v_{\text{beam}}$$

(`P_jet = 0.5 * T_thrust * v_beam`)

6.7 "Rendement total" électrique η_{elec} (nom et formule du code)

$$\eta_{\text{elec}} = \frac{P_{\text{jet}}}{P_{\text{total}}} \quad (\text{si } P_{\text{total}} > 0 \text{ sinon } 0)$$

(`eta_elec = P_jet/P_total if P_total>0 else 0`)

Interprétation (strictement selon les variables du code) :

- numérateur : puissance cinétique associée à la poussée et à la vitesse de faisceau (définie via $0.5Tv$),
- dénominateur : puissance faisceau + puissance de décharge imposée.

6.8 Rendements de masse (2 définitions dans le code)

Le code définit deux rendements en s'appuyant sur $\dot{N}_n$, $\dot{N}_{\text{beam}}$, $\dot{N}_{\text{wall}}$:

$$\eta_{\text{mass,total}} = \frac{\dot{N}_{\text{beam}} + \dot{N}_{\text{wall}}}{\dot{N}_n}$$

(`self.eta_mass_total = (N_dot_beam + N_dot_wall)/N_dot_neutral`)

$$\eta_{\text{mass,beam}} = \frac{\dot{N}_{\text{beam}}}{\dot{N}_n}$$

(`self.eta_mass_beam = N_dot_beam/N_dot_neutral`)

Différence (commentaires du code) :

- $\eta_{\text{mass,total}}$: *tous les ions produits* (ceux extraits + ceux perdus aux parois).
- $\eta_{\text{mass,beam}}$: *seulement les ions utiles au faisceau* (contribuent à la poussée).

7) Facteur limitant (logique affichée)

Le code affiche un texte selon la comparaison :

$$\text{si } I_{CL} < \tau I_{\text{plasma,available}} \Rightarrow \text{limitation “Extraction (I_CL)”}$$

sinon :

$$\text{limitation “Production (I_plasma)”}$$

(`if self.I_CL < self.I_plasma_available*self.tau: ...`)

8) Remarques importantes (issues du code, sans ajouter de calcul)

- Le modèle fixe `n_i = 0.01 * n_a` : ce n'est pas une équation plasma résolue, mais une hypothèse codée.
- `self.eta_mass` est initialisé à 0.0 dans `__init__` mais les rendements utilisés ensuite sont `eta_mass_total` et `eta_mass_beam`.
- Il n'y a pas d'autre rendement (pas de $\eta_{current}$, pas de $\eta_{voltage}$, etc.) dans la classe.